production differs from the latest code deployed to a main branch, so you still need to verify if the code you currently have is compatible with the changes deployed to production. The can-i-deploy tool works by checking the individual entries from the matrix table and checking if the consumer or provider version you're about to deploy is compatible with the other versions deployed to an environment, as listed in table 10.1.

Table 10.1 How can-i-deploy works

Consumer Version	Provider Version	Verification Status
12	40	Failed
13	41	Failed
13	42 (production)	Passed
14	42 (production)	Failed

Looking at table 10.1, we can easily visualize which consumer and provider versions are compatible. If provider version 42 has been deployed to production, we know that consumer version 13 can be deployed too, but not version 14 because it's not compatible with the version being used in production. To implement can-i-deploy in our contract testing workflow, we'll add two new scripts in our package.json file:

```
"scripts": {
   ...
   "can:i:deploy:consumer": "pact-broker
can-i-deploy --pacticipant WebConsumer
--version=$GITHUB_SHA --to-environment production",
   "can:i:deploy:provider": "pact-broker
can-i-deploy --pacticipant MoviesAPI
--version=$GITHUB_SHA --to-environment production"
   ...
}
```

The `can:i:deploy:consumer` verifies if the WebConsumer's current version is safe to be deployed to production, while the `can:i:deploy:provider` verifies if the provider version can be deployed to production. Next, let's add these two new commands as additional steps to our contract testing workflow by updating the contract-test-sample.yml file: